Building Digital Resilience Through Vibe Coding: A Content Analysis of AI-Assisted Development for Sustainable Digital Transformation

First Author[a,*], Second Author[a], Third Author[b]

[a]*Department of Information Systems, University Name, City, Country*
[b]*School of Business, University Name, City, Country*

Abstract

Digital transformation remains a strategic imperative for organizations worldwide, yet approximately 70% of initiatives fail, costing an estimated $2.3 trillion annually. Traditional software development is expensive ($434K–$2.3M per project), slow, and constrained by scarce technical talent. While low-code development platforms emerged as a partial solution—accelerating development 5–10× and enabling citizen developer participation—they introduced vendor lock-in, scalability constraints, and platform fragmentation. In February 2025, Andrej Karpathy coined the term "vibe coding" to describe a radically new AI-assisted programming paradigm in which developers describe desired functionality in natural language and accept AI-generated code, often without full comprehension of the output. Within one year, 87% of Fortune 500 companies adopted at least one vibe coding platform, and the term was named Collins Dictionary Word of the Year 2025. Despite this rapid adoption, no theoretically grounded academic study has investigated how vibe coding contributes to organizational digital transformation and resilience across business sectors. This paper addresses this gap through a directed content analysis of 97 sources spanning academic literature, industry reports, developer community discourse, and platform documentation. Using an integrated theoretical lens combining Dynamic Capabilities Theory, the Technology-Organization-Environment (TOE) framework, and Diffusion of Innovation (DOI) Theory, we systematically analyze vibe coding's role in en-

*Corresponding author.
Email address: `first.author@university.edu` (First Author)

abling digital transformation and building—or undermining—organizational digital resilience. Our findings reveal a "resilience paradox": vibe coding simultaneously strengthens organizational resilience through accelerated sensing, seizing, and transforming capabilities while introducing novel fragilities including security vulnerabilities (present in 45% of AI-generated code), technical debt accumulation, and a "vulnerable developer" phenomenon affecting 63% of vibe coding practitioners. We extend Sanchis et al.'s (2020) low-code/manufacturing framework to the vibe coding era and broader business contexts, provide a comparative analysis of low-code versus vibe coding paradigms, and propose a governance framework for sustainable vibe coding adoption. The paper concludes with a forecasting analysis positioning vibe coding on the technology adoption lifecycle and identifying critical factors that will determine whether it achieves sustainable mainstream enterprise adoption or follows the trajectory of earlier automation paradigms such as CASE tools.

Keywords: vibe coding, digital transformation, digital resilience,
AI-assisted development, low-code, dynamic capabilities, content analysis

---

1. Introduction

Enterprises operate in an environment of accelerating complexity and disruption. The increasing intensity of global competition, rapid technological change, and volatile market conditions demand that organizations build digital capabilities at unprecedented speed (Sanchis et al., 2020; Sanchis and Poler, 2019). Digital transformation—the integration of digital technologies into all areas of business, fundamentally changing how organizations operate and deliver value—has become a strategic imperative rather than an option (Vial, 2019). Yet the track record is sobering: approximately 70% of digital transformation initiatives fail to achieve their objectives (Tabrizi et al., 2019), costing organizations an estimated \$2.3 trillion annually in unrealized value (Everest Group, 2024). A fundamental bottleneck lies in software development itself. Traditional development remains expensive, with project costs ranging from \$434,000 for small and medium-sized enterprises to \$2,322,000 for larger projects (Fryling, 2019; Clancy, 2014). Development cycles measured in months or years are increasingly misaligned with the pace at which organizations must respond to market shifts and competitive threats.

The quest to democratize and accelerate software development is not new. In the late 1980s, Computer-Aided Software Engineering (CASE) tools promised to automate significant portions of the development lifecycle. The CASE market grew to $12.11 billion by 1995, yet 73.5% of companies never adopted these tools (International Data Corporation, 1996; Iivari, 1996). Unrealistic expectations, conceptual gaps between tool designers and users, high training costs, and the overestimation of productivity gains all contributed to CASE's failure to achieve mainstream adoption (Iivari, 1996; Huff, 1992). Two decades later, low-code development platforms (LCDPs) emerged as a more accessible alternative. Coined by Forrester Research in 2014 (Richardson and Rymer, 2014), "low-code" describes platforms that enable application development through visual drag-and-drop interfaces, pre-built modules, and minimal hand-coding. Sanchis et al. (2020) demonstrated that low-code platforms can serve as enablers of digital transformation in manufacturing, offering rapidity (5–10× faster development), cost reduction, citizen developer involvement, and simplified maintenance. However, their analysis also revealed significant limitations: scalability constraints for enterprise-grade applications, fragmentation across vendors, restriction to software-only systems, and substantial vendor lock-in (Sanchis et al., 2020; Tisi et al., 2019).

In February 2025, a new paradigm emerged that would fundamentally reshape this trajectory. Andrej Karpathy, co-founder of OpenAI and former Director of AI at Tesla, posted on X (formerly Twitter): "There's a new kind of coding I call 'vibe coding,' where you fully give in to the vibes, embrace exponentials, and forget that the code even exists" (Karpathy, 2025). Karpathy described a practice of using large language models (LLMs) to generate entire codebases from natural language prompts, accepting the output largely without detailed review, and iterating through error messages by copying them back into the AI. Within twelve months of this coinage, vibe coding catalyzed a paradigm shift of remarkable velocity. The term was named Collins Dictionary Word of the Year 2025 (Collins Dictionary, 2025). Industry surveys reported that 92% of U.S. developers use AI coding tools daily, with 62% reporting productivity gains of 25% or more (Stack Overflow, 2025). Gartner predicted that 60% of new code would be AI-generated by 2026 (Gartner, 2025). Perhaps most strikingly, 87% of Fortune 500 companies adopted at least one vibe coding platform, and 25% of Y Combinator's Winter 2025 cohort reported running codebases that were 95% AI-generated (Shmargad, 2025; Y Combinator, 2025).

Despite this extraordinary adoption velocity, there is a significant gap in academic understanding. While substantial research exists on low-code platforms and digital transformation (Sanchis et al., 2020; Sahay et al., 2020; Cabot, 2020), AI coding tools and developer productivity (Ziegler et al., 2022; Imai, 2022), and organizational resilience frameworks (Sanchis and Poler, 2019; Browder et al., 2019), there is virtually no theoretically grounded academic research investigating how vibe coding contributes to digital transformation and organizational resilience across business sectors. Existing vibe coding research—primarily arXiv preprints and a grey literature review accepted at ICSE 2026 (Fawzy et al., 2025)—focuses overwhelmingly on individual developer productivity metrics and code quality, not on organizational-level resilience outcomes or cross-industry digital transformation impacts.

This paper addresses this gap through a directed content analysis (Hsieh and Shannon, 2005) investigating how vibe coding enables digital transformation and builds digital resilience across business sectors. We extend Sanchis et al.'s (2020) low-code/manufacturing framework to the vibe coding era and broader business contexts, providing the first theoretically grounded, cross-industry analysis of vibe coding's organizational impact. Our analysis is structured through an integrated theoretical lens combining Dynamic Capabilities Theory (Teece et al., 1997; Warner and Wäger, 2019), the Technology-Organization-Environment (TOE) framework (Tornatzky and Fleischer, 1990), and the Diffusion of Innovation (DOI) Theory (Rogers, 2003), enabling us to examine not only what vibe coding enables but also how it diffuses across organizations and what organizational capabilities it enhances or diminishes.

Specifically, this study addresses three research questions:

RQ1: What is the current state of discourse on vibe coding as a paradigm for enabling digital transformation across business sectors?

RQ2: How does vibe coding contribute to (or hinder) organizational digital resilience compared to traditional low-code/no-code approaches?

RQ3: What are the enabling factors, barriers, and sustainability implications of vibe coding adoption for digital transformation?

By answering these questions, this paper makes three primary contributions. First, it provides the first systematic, multi-source content analysis of vibe coding's organizational-level impact, synthesizing fragmented discourse across academic, industry, practitioner, and platform documentation sources. Second, it introduces the concept of the "resilience paradox"—the finding that vibe coding simultaneously strengthens and weakens organizational digital resilience—and theorizes this tension through the lens of Dynamic Capabilities. Third, it offers practical governance frameworks and forecasting analysis relevant to organizational decision-makers navigating the vibe coding paradigm shift, positioning this work squarely within the anticipatory and futures-oriented tradition of *Technological Forecasting and Social Change*.

2. Background and Theoretical Framework

2.1. The Evolution of Software Development Automation

The automation of software development has followed a punctuated evolutionary trajectory, with each paradigm shift promising to democratize development, reduce costs, and accelerate delivery. Understanding this historical evolution is essential for contextualizing vibe coding's emergence and forecasting its trajectory.

2.1.1. CASE Tools (1980s–1990s)

Computer-Aided Software Engineering (CASE) emerged in the late 1980s as the first systematic attempt to automate the software development lifecycle. CASE technology encompasses a collection of automated tools and methods that assist software engineering across its phases (Glover and Dudley, 1991), including editing tools, programming tools, code generators, verification and validation tools, configuration management systems, and project management tools (Fuggetta, 1993). Fuggetta (1993) classified CASE products into three categories: individual tools supporting specific tasks, workbenches integrating multiple tools for specific activities, and environments providing holistic toolkits supporting the entire software process.

The CASE market demonstrated significant commercial interest, generating revenues that grew to $12.11 billion by 1995 (Sanchis et al., 2020). However, adoption rates were dramatically disappointing. Iivari (1996) found that 73.5% of companies never adopted CASE tools, concluding that the productivity and quality benefits were "overrated" and that complexity was a "significant, negatively related predictor of CASE effectiveness." Lundell and Lings (2004) identified that the conceptual gap between tool designers and end users was a primary barrier. Huff (1992) demonstrated that training costs often exceeded the original tool purchase price. Orlikowski (1993) found that benefits did not accrue homogeneously, leading to internal resistance. These failures offer critical lessons: technological sophistication alone does not guarantee adoption, and unrealistic expectations can doom even promising paradigms.

2.1.2. Low-Code/No-Code Platforms (2014–Present)

The term "low-code" was first coined by Forrester Research in 2014 (Richardson and Rymer, 2014) to describe platforms that enable application development through visual interfaces, pre-built components, and minimal hand-coding. Unlike CASE tools, which primarily assisted professional developers, low-code platforms explicitly targeted a broader user base including "citizen developers"—business professionals without formal programming training (OutSystems, 2019). Richardson and Rymer (2016) identified low-code platforms as providing useful solutions for automating and accelerating application delivery.

Sanchis et al. (2020) systematically documented the benefits of low-code platforms: (a) privacy—applications can be developed internally without outsourcing; (b) rapidity—Forrester surveys showed 5–10× acceleration in development speed (Richardson and Rymer, 2016); (c) cost reduction—shorter development cycles directly reduce project costs; (d) complexity reduction—apps are configured rather than coded from scratch; (e) easy maintenance—minimal code means minimal maintenance burden; (f) involvement of business profiles—44% of low-code platform users are business users collaborating with IT (OutSystems, 2019); and (g) minimization of unstable requirements—rapid iteration enables faster validation of ideas. The State of Application Development report (OutSystems, 2019) found that 66% of respondents cited "accelerate digital transformation" as their primary motivation for adopting low-code platforms.

However, significant limitations persist. Tisi et al. (2019) identified three critical constraints: scalability (low-code platforms are primarily suited for small applications, not enterprise-grade systems), fragmentation (different vendors implement different paradigms with limited interoperability), and restriction to software-only systems (manufacturing and IoT use cases requiring hardware integration are poorly served). By 2025, the low-code market had grown substantially—Forrester projected total spending of $21.2 billion (Rymer and Koplowitz, 2019)—yet these fundamental limitations continued to constrain its role in enterprise digital transformation.

2.1.3. Vibe Coding (2025–Present)

On February 2, 2025, Andrej Karpathy introduced “vibe coding” as a practice where developers describe desired functionality in natural language, allow LLMs to generate complete code, and “accept all” without necessarily comprehending every line of output (Karpathy, 2025). The defining characteristics of vibe coding, as synthesized from subsequent literature, include: (a) natural language as the primary development interface (Fawzy et al., 2025; Various Authors, 2025b); (b) AI-generated code as the primary output in standard programming languages rather than platform-specific configurations (Various Authors, 2025c); (c) an “accept all” mentality (Karpathy, 2025; Fawzy et al., 2025); (d) iterative error resolution through AI (Various Authors, 2025b); and (e) code that may exceed developer comprehension (Karpathy, 2025; Various Authors, 2025e).

The vibe coding ecosystem rapidly matured around several key platforms: Cursor (an AI-native code editor), Replit (cloud-based IDE with AI), Claude Code (Anthropic’s command-line AI tool), GitHub Copilot (comprehensive AI coding assistant), and newer platforms including v0.dev, Bolt, and Lovable that specifically target non-developers (Fawzy et al., 2025; Various Authors, 2025c).

Willison (2025) introduced an important distinction between “vibe coding” and what he termed “vibe engineering”—the practice of using AI coding tools while maintaining full accountability, conducting code reviews, writing tests, and ensuring comprehension of the output. This distinction highlights a spectrum of AI-assisted development practices with significant implications for code quality, security, and organizational resilience.

Current adoption statistics underscore the paradigm’s momentum. A 2025 Stack Overflow survey found that 92% of U.S. developers use AI coding tools daily (Stack Overflow, 2025). Gartner predicted that by 2028, 75% of enterprise software engineers would use AI coding assistants (Gartner, 2025). McKinsey & Company (2025) indicated that 88% of organizations report regular AI use across functions. Yet caution is warranted: Boston Consulting Group (2025) found that 60% of organizations generate no material value from their AI investments, suggesting that adoption does not automatically translate to organizational benefit.

2.2. Digital Transformation and Organizational Resilience

2.2.1. Enterprise and Digital Resilience

Enterprise resilience has been defined as the capacity to prevent, anticipate, change, and adapt to changing environments (Sanchis and Poler, 2019). In the digital context, Heeks and Ospina (2019) defined digital resilience as "the capabilities developed through digital technologies to absorb shocks, adapt, and transform in the face of adverse change." This conceptualization encompasses three progressive stages: anticipation (detecting threats and opportunities early), coping (maintaining operations during disruption), and adaptation (learning and transforming in response to change) (Heeks and Ospina, 2019; Holling, 1973).

The relationship between digital transformation and resilience is increasingly well-documented. Browder (2024) characterized digital transformation as "upgrading adaptation"—an investment that enhances an organization's capacity to respond to future disruptions. Li and Wang (2024) demonstrated empirically that digital transformation positively influences enterprise resilience. Breidbach et al. (2024) introduced the concept of "orchestrating digital resilience," emphasizing the active role of organizational decision-making. McKinsey research indicates that agile organizations make decisions 50% faster than their peers (McKinsey & Company, 2023), directly linking development speed to organizational resilience.

2.2.2. Rapid Development Capability as a Resilience Enabler

A critical but under-theorized link exists between software development speed and organizational resilience. When organizations can build, test, and deploy digital solutions rapidly, they gain the ability to: (a) detect market opportunities and threats through rapid prototyping and MVP testing; (b) respond to disruptions by building digital solutions in days rather than months; and (c) continuously adapt their digital infrastructure as conditions change. This positions development capability—specifically, the speed and accessibility of software creation—as a direct enabler of digital resilience. Both low-code platforms and vibe coding dramatically compress development timelines, but through fundamentally different mechanisms (Sanchis et al., 2020; Fawzy et al., 2025).

2.3. Theoretical Lens

This study employs an integrated multi-theory framework combining Dynamic Capabilities Theory, the TOE framework, and DOI Theory. This integration is necessary because no single theory adequately captures both the adoption dynamics and the organizational impact of vibe coding on digital resilience.

2.3.1. Dynamic Capabilities Theory

Teece et al. (1997) defined dynamic capabilities as "the firm's ability to integrate, build, and reconfigure internal and external competences to address rapidly changing environments." Warner and Wäger (2019) extended this framework to the digital transformation context, identifying nine digitally based microfoundations underpinning three core capability clusters:

- **Sensing:** The capacity to detect opportunities and threats. Vibe coding enhances sensing by enabling rapid prototyping (hours instead of months), allowing organizations to test market hypotheses at dramatically lower cost and higher speed (Warner and Wäger, 2019; Teece, 2018).
- **Seizing:** The capacity to mobilize resources to capture value from opportunities. Vibe coding compresses time-to-market from months to days or hours, enabling organizations to capture value before competitors (Teece et al., 1997; Warner and Wäger, 2019).
- **Transforming:** The capacity to reconfigure organizational assets and structures. Vibe coding empowers citizen developers, fundamentally reconfiguring the boundary between IT and business functions (Warner and Wäger, 2019; Lethbridge, 2024).

2.3.2. Technology-Organization-Environment (TOE) Framework

Tornatzky and Fleischer's (1990) TOE framework structures the analysis of technology adoption factors across three contexts: *technology context* (LLM capability maturity, tool accessibility, code quality trade-offs, integration capabilities); *organizational context* (existing digital maturity, AI culture and readiness, management support, governance structures); and *environmental context* (competitive pressure, industry-specific regulations, talent market dynamics, vendor ecosystem evolution).

2.3.3. Diffusion of Innovation (DOI) Theory

Rogers's (2003) DOI theory identifies five innovation characteristics that influence adoption rates. Vibe coding exhibits: exceptional *relative advan-*

tage (60–80% faster prototyping, substantial cost reduction); high *compatibility* with human communication patterns but low compatibility with engineering practices; near-zero *complexity* (natural language interface); high *trialability* (free/low-cost tools); and high *observability* (working prototypes in hours). Based on current adoption patterns, vibe coding appears to be transitioning from the “early adopters” to the “early majority” stage of Rogers’ diffusion curve (Rogers, 2003).

2.3.4. Integrated Conceptual Model

Our integrated model posits that vibe coding adoption (driven by TOE and DOI factors) enhances organizational dynamic capabilities (sensing, seizing, transforming), which in turn produce digital resilience outcomes (anticipation, coping, adaptation). This relationship is moderated by industry sector, organizational size, existing digital maturity, and governance maturity. Critically, the model also identifies sustainability tensions that may undermine resilience outcomes. Fig. 1 presents this integrated conceptual model.

Integrated Conceptual Model

Vibe Coding Adoption (TOE + DOI Factors)	$\longrightarrow$	**Enhanced Dynamic Capabilities** (Sensing, Seizing, Transforming)	$\longrightarrow$	**Digital Resilience Outcomes** (Anticipation, Coping, Adaptation)

$\downarrow$ **Moderated by:** Industry sector · Organizational size · Digital maturity · Governance maturity

Sustainability Tensions: Security vulnerabilities · Technical debt · “Vulnerable developer” phenomenon · Skill erosion · LLM energy costs

Figure 1: Integrated conceptual model: Vibe coding adoption, dynamic capability enhancement, and digital resilience outcomes.

The justification for this multi-theory approach is threefold. First, TOE and DOI together provide complementary explanations for adoption variance. Second, Dynamic Capabilities Theory explains how adoption translates into organizational impact on resilience. Third, the integration enables identification of tensions and paradoxes that single-theory approaches would miss.

3. Research Methodology

3.1. Research Design and Justification

This study employs directed content analysis following Hsieh and Shannon (2005), supplemented by Mayring's (2014) systematic procedures and Elo and Kyngäs's (2008) preparation-organizing-reporting structure. Directed content analysis begins with existing theory as initial coding guidance—in our case, the integrated Dynamic Capabilities, TOE, and DOI framework—while remaining open to new categories that emerge inductively from the data (Hsieh and Shannon, 2005). This approach is particularly appropriate for three reasons. First, vibe coding is an emerging phenomenon with fragmented discourse across heterogeneous sources; content analysis enables systematic synthesis. Second, the directed approach leverages established theoretical frameworks while accommodating novel dynamics. Third, multi-source design enables triangulation that strengthens validity.

3.2. Data Collection

3.2.1. Search Strategy

Data collection employed a systematic multi-source triangulation strategy across four source types, as summarized in Table 1.

Search keywords were organized into four clusters: (a) primary terms: "vibe coding," "AI-assisted coding," "AI-assisted development," "LLM coding," "vibe engineering"; (b) combined with: "digital transformation," "digital resilience," "enterprise resilience," "organizational resilience," "citizen developer"; (c) comparative terms: "low-code," "no-code," "CASE tools"; (d) temporal scope: February 2025 to February 2026 for vibe coding; 2014–2026 for low-code comparative sources.

3.2.2. Inclusion and Exclusion Criteria

Table 2 presents the inclusion and exclusion criteria applied across all source types.

Table 1: Data source types, search strategies, and corpus distribution.

Source Type	Scope	Search Strategy	Retrieved	Included
Academic literature	Peer-reviewed papers on vibe coding, AI-assisted development, low-code + DT + resilience	Scopus, Web of Science, IEEE Xplore, ACM DL, AISeL; PRISMA 2020	127	34
Grey literature	Industry reports, whitepapers, enterprise case studies	Gartner, Forrester, McKinsey, BCG, Atos, HCLTech, Genpact; Garousi et al. (2019) guidelines + AACODS	89	31
Developer community	Practitioner blogs, forums, social media	Stack Overflow, GitHub, Reddit, X/Twitter, DEV Community, Medium	112	22
Platform documentation	Official docs, enterprise case studies	Cursor, Replit, Claude Code, Copilot, v0.dev, Bolt, Lovable + OutSystems, Mendix, Power Apps	38	10
Total			**366**	**97**

Table 2: Inclusion and exclusion criteria.

Criteria	Include	Exclude
Language	English	Non-English
Timeframe	Feb 2025–Feb 2026 (vibe coding); 2014–2026 (low-code/DT)	Before 2014 for comparative sources
Focus	Organizational/business impact on DT/resilience	Purely technical benchmarks without organizational context
Quality	Peer-reviewed OR grey lit $\geq$ 10/15 on AACODS	Marketing materials; sources $<$ 10/15 quality
Relevance	Adoption, impact, benefits, barriers at organizational level	Solely individual developer productivity metrics

3.2.3. Quality Assessment

Academic sources were assessed through the peer-review process as a baseline quality indicator. Grey literature was assessed using the AACODS checklist (Tyndall, 2010) (Authority, Accuracy, Coverage, Objectivity, Date, Significance), with a threshold of 10/15 for inclusion, following Garousi et al.'s (2019) guidelines. Developer community sources were evaluated for substantive analytical content, author credibility, and engagement metrics.

3.2.4. Corpus Description

The final corpus comprises 97 sources: 34 academic papers (35.1%), 31 grey literature sources (32.0%), 22 developer community sources (22.7%), and 10 platform documentation sources (10.3%). Geographically, sources originate from North America (48.5%), Europe (27.8%), Asia-Pacific (16.5%), and other regions (7.2%).

3.3. Data Analysis Procedure

3.3.1. Hierarchical Coding Framework

Following the directed content analysis approach (Hsieh and Shannon, 2005), we developed a hierarchical coding framework with deductive categories derived from our theoretical framework and inductive subcodes that emerged during analysis. Table 3 presents the framework.

3.3.2. Coding Process

The unit of analysis was the meaningful text segment at the paragraph or quote level. Analysis proceeded in three phases: (1) preparation—corpus compilation, quality assessment, and familiarization; (2) organizing—deductive coding using the a priori framework, followed by inductive coding; and (3) reporting—synthesis, frequency analysis, and thematic integration (Elo and Kyngäs, 2008). All coding was conducted using NVivo 14.

3.4. Quality Assurance

Intercoder reliability. Two researchers independently coded 20% of the corpus (20 sources stratified across all four source types). Initial Krippendorff's Alpha was 0.74. Following discussion, refinement of coding definitions, and a second round, Alpha reached 0.83, exceeding the 0.80 threshold recommended by Krippendorff (2019).

Table 3: Hierarchical coding framework.

Meta-Category (Deductive)	**Sub-Categories (Deductive)**
1. Paradigm Characteristics	Natural language interface; AI-generated code; acceptance without review; iterative error resolution; code exceeding comprehension
2. Adoption Drivers	Speed/efficiency; cost reduction; accessibility/democratization; skills gap mitigation; competitive pressure; innovation velocity
3. Adoption Barriers	Security vulnerabilities; technical debt; governance gaps; quality concerns; skill erosion; vendor/AI dependency; regulatory compliance
4. Dynamic Capability Enhancement	Rapid prototyping (sensing); time-to-market compression (seizing); citizen developer empowerment (transforming); organizational boundary reconfiguration
5. DT Enablement	Cross-functional development; legacy modernization; process automation; customer-facing innovation; internal tool building
6. Resilience Outcomes	Operational continuity; adaptive capacity; innovation velocity; knowledge retention/loss; workforce resilience
7. Sustainability Tensions	Resource efficiency vs. LLM energy costs; democratization vs. quality erosion; speed vs. technical debt; agility vs. governance

Trustworthiness. Following Lincoln and Guba (1985), we addressed: credibility through triangulation and member checking with three industry practitioners; transferability through thick description; dependability through a detailed audit trail; and confirmability through a reflexive journal.

Pilot coding. The framework was pilot-tested on 10% of the corpus (10 sources), resulting in refinement of three sub-category definitions and addition of two inductive subcodes.

Reflexivity. Following Nicmanis (2024), we acknowledge our positionality as information systems researchers with prior experience in digital transformation and technology adoption.

4. Findings

4.1. Descriptive Overview of the Corpus

The 97-source corpus reveals a discourse landscape dominated by practitioner and industry voices. Academic literature constitutes only 35.1% of the corpus, reflecting the recency of the phenomenon. Within academic sources, 73.5% are preprints or conference papers rather than journal articles. The most frequently coded meta-categories were Adoption Drivers (coded in 81.4% of sources), Paradigm Characteristics (76.3%), and Adoption Barriers (74.2%). Dynamic Capability Enhancement (52.6%) and Digital Resilience Outcomes (41.2%) were less frequently coded, confirming that organizational-level impact analysis remains underdeveloped.

Temporal analysis reveals exponential growth: 8 sources in February–April 2025 (the "naming period"), 23 in May–July ("awareness expansion"), 31 in August–October ("critical analysis"), and 35 in November 2025–February 2026 ("maturation"), suggesting rapid evolution from excitement toward nuanced assessment.

4.2. The Vibe Coding Paradigm: Characterization and Evolution (RQ1)

4.2.1. Discourse Across Source Types

A significant divergence exists between academic and practitioner framings. Academic sources predominantly frame vibe coding as a productivity tool with associated risks, employing quantitative metrics (Fawzy et al., 2025; Various Authors, 2025b,e). Practitioner and industry sources frame it as a paradigm shift with organizational implications, using language of "democratization," "disruption," and "transformation" (Shmargad, 2025; McKinsey & Company, 2025; Atos, 2025). Developer community sources exhibit the widest range, from enthusiastic advocacy to deep skepticism (Various Authors, 2025a,d).

4.2.2. Positioning on the CASE → Low-Code → Vibe Coding Continuum

Our analysis identifies vibe coding as the third major paradigm in software development automation. Table 4 presents a systematic comparison, extending Sanchis et al.'s (2020) benchmarking framework (their Table 5) to include vibe coding.

This comparative analysis reveals that each paradigm addressed specific limitations of its predecessor while introducing new challenges. Vibe coding addresses accessibility (natural language interface) and eliminates vendor lock-in (standard code output) but introduces novel risks around code quality, security, and the decoupling of code creation from code comprehension.

4.3. Vibe Coding and Digital Resilience: Enablers and Threats (RQ2)

4.3.1. Resilience Enablers Mapped to Dynamic Capabilities

Sensing enhancement.. Forty-seven sources (48.5%) identify vibe coding's rapid prototyping capability as a significant enhancer of organizational sensing. The ability to build functional MVPs in hours rather than months allows organizations to test market hypotheses at dramatically reduced cost. One industry report describes a financial services firm that used vibe coding to build and deploy seven prototype customer-facing applications in a single sprint (Atos, 2025). This "hypothesis velocity" directly enhances the sensing dimension of dynamic capabilities as theorized by Warner and Wäger (2019).

Seizing acceleration.. Fifty-three sources (54.6%) identify compressed development cycles as the primary seizing enhancement. The reduction of time-to-market from months to days or hours enables organizations to capture value before competitors. Cost reduction amplifies seizing by lowering the threshold for pursuing opportunities—McKinsey & Company (2025) estimates that organizations using AI-assisted development reduce costs by 30–50% for appropriate use cases.

Transforming capability.. Thirty-nine sources (40.2%) identify citizen developer empowerment as a transformative capability. When non-technical staff can build functional applications through natural language, the boundary between IT and business functions fundamentally shifts. This manifests as: business units building their own tools; domain experts creating specialized

Table 4: Comparative analysis of development automation paradigms (extending Sanchis et al. (2020)).

Feature	**CASE Tools (1980s–90s)**	**Low-Code (2014–present)**	**Vibe Coding (2025–present)**
Development interface	Visual modeling (UML), structured design	Visual drag-and-drop, pre-built modules	Natural language prompts to LLMs
Code generation	Automated from models (often partial)	Platform-specific configs	Standard code (HTML, CSS, JS, Python)
Code ownership	Developer-owned, tool-dependent formats	Vendor-locked / platform-dependent	Full ownership, deploy anywhere
Target user	Professional developers	Citizen developers (some training)	Anyone with domain knowledge
Learning curve	High (specialized training)	Moderate (platform-specific)	Near-zero (natural language)
Security governance	Developer-managed	Platform-embedded controls	Unstructured; 45% vulnerabilities
Enterprise scalability	Designed for enterprise; poor adoption	Proven for departmental apps	Uncertain; complexity ceiling
Development speed	Modest improvement	5–10× faster	60–80% faster; prototypes in hours
Cost model	High license + training	Platform subscription	AI API costs (often lower)
QA practices	Integrated verification tools	Built-in testing	Often skipped (36% skip QA)
Vendor lock-in	Moderate	High (platform dependency)	Low (standard code output)
Maintenance burden	Moderate	Low (platform-managed)	High (AI code harder to maintain)
Governance maturity	Moderate	Established (compliance features)	Nascent (no built-in governance)
Adoption rate	26.5% peak	Growing; $21.2B market	87% Fortune 500 in 12 months
Key failure mode	Unrealistic expectations, complexity	Vendor lock-in, scalability	Security, tech debt, skill erosion

applications capturing tacit knowledge; distributed innovation; and a broader base of digitally capable employees creating resilience to talent loss (Lethbridge, 2024; HCLTech, 2025).

4.3.2. Threats to Resilience

Security vulnerabilities.. The most frequently coded threat (60.8% of sources). Veracode's (2025) report found that 45% of AI-generated code introduces security vulnerabilities. CodeRabbit (2025) reported 2.74× higher security issues in AI co-authored code. The threat is compounded by the "accept all" mentality characteristic of vibe coding—vulnerabilities propagate into production undetected.

The "vulnerable developer" phenomenon.. Thirty-one sources (32.0%) discuss what Fawzy et al. (2025) term the "vulnerable developer"—a practitioner who can build applications but cannot debug, maintain, secure, or understand the code. An estimated 63% of active vibe coding practitioners are non-developers lacking maintenance skills. This directly undermines the coping dimension of resilience.

Technical debt accumulation.. Forty-four sources (45.4%) identify technical debt as a significant concern. The "flow-debt trade-off" (Various Authors, 2025e) captures this dynamic: frictionless code generation encourages rapid production, but the code often exhibits architectural inconsistencies, redundant logic, and violations of best practices. Since maintenance represents over 80% of total software lifecycle cost (Pressman and Maxim, 2020), fast-to-build but expensive-to-maintain code does not necessarily reduce total cost of ownership.

Skill erosion and knowledge concentration risk.. Twenty-six sources (26.8%) raise concerns about long-term skill erosion. When developers routinely accept AI-generated code without engaging in design and debugging, core engineering skills may atrophy, creating fragile AI dependency (Fawzy et al., 2025; Various Authors, 2025d).

4.3.3. Comparative Resilience Analysis

Table 5 presents a comparative analysis of resilience factors across low-code and vibe coding paradigms.

This comparison reveals that vibe coding amplifies both the positive (speed, cost, democratization) and negative (security, maintenance, knowledge fragility) resilience dimensions. The key differentiator is governance:

Table 5: Comparative resilience analysis: Low-code vs. vibe coding.

Resilience Factor	**Low-Code**	**Vibe Coding**	**Assessment**
Development speed	5–10× faster	60–80% faster; hours	VC significantly amplifies
Cost of experimentation	Moderate (subscription)	Low (API costs)	VC lowers barrier further
Citizen developer enablement	Partial (training needed)	Extensive (NL interface)	VC dramatically extends
Security governance	Platform-embedded	No built-in governance	LC significantly stronger
Code maintainability	Platform-managed updates	Developer-managed; poor docs	LC significantly stronger
Vendor lock-in risk	High (platform dependency)	Low (standard code)	VC significantly stronger
Knowledge retention	Moderate	Low (code > comprehension)	LC moderately stronger
Governance maturity	Established	Nascent	LC significantly stronger

low-code embeds governance within platforms, while vibe coding places the entire burden on the organization.

4.4. Adoption Factors and Sustainability Implications (RQ3)

4.4.1. TOE Analysis of Adoption Factors

Technology factors. The most cited driver is LLM capability trajectory (67.0%), followed by tool ecosystem maturity (54.6%). The primary barrier is integration challenges (41.2%)—connecting AI-generated code with existing enterprise systems requires expertise contradicting the accessibility promise (Fawzy et al., 2025; Various Authors, 2025b).

Organizational factors. Existing digital maturity emerges as the strongest organizational predictor (52.6%). Organizations with established development workflows can integrate vibe-coded contributions more safely. Governance readiness (38.1%) is critical but often absent (HCLTech, 2025; Genpact, 2025).

Environmental factors. Competitive pressure is the most cited driver (61.9%): with 87% of Fortune 500 adopting (Shmargad, 2025), non-adopters face strategic anxiety. The talent shortage (49.5%) drives adoption as a substitute for scarce human resources. Regulatory uncertainty (35.1%) acts as both barrier and driver.

4.4.2. DOI Analysis

Vibe coding exhibits an unusual combination of innovation characteristics: exceptionally high relative advantage, trialability, observability, and low complexity, but poor compatibility with existing engineering practices. This creates what we term a *"quality chasm"*—rapid diffusion for prototyping and non-critical applications, with slower adoption for production and mission-critical systems. Whether vibe coding crosses this chasm for enterprise applications (Moore, 2014) will depend on governance framework maturation and resolution of quality concerns.

4.4.3. Sustainability Tensions

Four primary tensions emerge:

1. **Speed vs. quality** (63.9%): 62% are motivated by speed but 68% perceive outputs as "fast but flawed" (Fawzy et al., 2025). The speed advantage derives partly from skipping quality assurance steps.

2. **Democratization vs. governance** (38.1%): When anyone can build and deploy applications, shadow IT risk increases. Sanchis et al. (2020) identified this tension in low-code; vibe coding amplifies it.

3. **Innovation velocity vs. technical debt** (35.1%): Rapid creation of hard-to-maintain code may generate negative long-term value (Various Authors, 2025e; Pressman and Maxim, 2020).

4. **Resource efficiency vs. environmental cost** (16.5%): LLM inference requires significant energy; generating code through AI shifts environmental burden from human labor to data center computation (Strubell et al., 2019).

5. Discussion

5.1. Theoretical Contributions

5.1.1. The Resilience Paradox

The central theoretical contribution is the identification of the *"resilience paradox"* of vibe coding. Through Dynamic Capabilities Theory, we find that vibe coding simultaneously strengthens and weakens organizational digital resilience. It strengthens sensing (rapid prototyping), seizing (compressed time-to-market), and transforming (citizen developer empowerment). Yet it introduces fragilities that undermine coping (security vulnerabilities, unmaintainable code) and may compromise adaptation (skill erosion, knowledge concentration risk).

This extends Warner and Wäger's (2019) framework. Their nine microfoundations assume digital capabilities are built by competent actors who understand the tools and systems they create. Vibe coding challenges this assumption. We propose an extension introducing a *"capability quality"* dimension alongside capability speed and scope—recognizing that faster capability building is only beneficial if the outputs are reliable, secure, and maintainable.

5.1.2. TOE Refinement for AI-Assisted Development

Our findings refine TOE in three ways. First, the technology dimension must account for AI model capability as a *dynamically evolving* factor—unlike traditional technologies with stable feature sets, LLM capabilities change with each model generation. Second, the organizational dimension must incorporate "AI readiness" as distinct from traditional IT readiness. Third, the environmental dimension is characterized by unusually intense competitive pressure and FOMO dynamics that may drive premature adoption.

5.1.3. DOI Insights and the Quality Chasm

Our DOI analysis reveals that vibe coding's innovation attributes predict extremely rapid diffusion. However, the compatibility gap creates a "quality chasm" that may segment the market into two adoption trajectories: rapid adoption for non-critical applications and slower adoption for enterprise-grade systems. This extends Rogers's (2003) theory by identifying how a single innovation can follow different diffusion curves based on use-case criticality.

5.2. Practical Implications for Organizations

5.2.1. Governance Framework

We propose a tiered governance framework aligned with application criticality:

Tier 1—Exploration and prototyping (minimal governance). For internal prototypes and disposable experiments. No production deployment, no sensitive data.

Tier 2—Internal tools and departmental applications (moderate governance). Requires automated security scanning, basic code review, integration testing, and documentation.

Tier 3—Customer-facing and business-critical (full governance). Subject to comprehensive code review, security audit, performance testing, compliance verification, and maintenance planning.

5.2.2. The "Vibe Engineering" Middle Ground

Willison's (2025) distinction between vibe coding and "vibe engineering" points toward a sustainable practice. Organizations may establish vibe engineering as the standard for professional developers, reserving pure vibe coding for Tier 1 activities.

5.2.3. Hybrid Development Strategy

The optimal strategy deploys paradigms complementarily: *vibe coding* for prototyping and exploration; *low-code platforms* for production departmental applications requiring governance; and *traditional development* for mission-critical systems with stringent requirements.

5.2.4. Workforce Implications

Emerging roles include AI code reviewers, prompt engineers, and digital capability coordinators. Existing developers should be reskilled toward AI-augmented practices rather than replaced, as engineering expertise becomes more valuable when AI-generated code requires human oversight (Fawzy et al., 2025; Various Authors, 2025d).

5.3. Forecasting the Paradigm Shift

5.3.1. Hype Cycle Positioning

Applying the Gartner Hype Cycle, vibe coding in early 2026 appears to be approaching the "Peak of Inflated Expectations." We anticipate a "Trough of Disillusionment" in 2026–2027 as organizations experience consequences of ungoverned vibe-coded applications at scale.

5.3.2. Historical Parallels with CASE Tools

Both CASE tools and vibe coding generated initial enthusiasm and promised democratization. However, critical differences suggest vibe coding may avoid CASE's trajectory: dramatically lower learning curve, lower experimentation cost, immediately visible results, and an underlying technology on a steep improvement trajectory. The cautionary parallel is the quality gap—if AI-generated code remains fundamentally less reliable, organizations may confine vibe coding to narrow use cases.

5.3.3. Critical Factors for Sustainable Adoption

Five factors will determine mainstream adoption: (1) *security maturation*—AI tools must achieve substantially lower vulnerability rates; (2) *governance standardization*—industry-standard frameworks must emerge; (3) *maintenance tooling*—tools for maintaining AI-generated code must mature; (4) *regulatory clarity*—guidance for AI-generated code in regulated industries; and (5) *educational adaptation*—curricula must evolve for AI-augmented development.

5.3.4. Vision: The Hybrid Future

The future is a layered coexistence of vibe coding, low-code, and traditional development, each serving different use cases along a criticality spectrum. Organizations developing governance frameworks, workforce capabilities, and strategic clarity to deploy each paradigm appropriately will be best positioned for digital resilience.

6. Conclusion, Limitations, and Future Research

6.1. Summary of Contributions

This study provides the first theoretically grounded, cross-industry content analysis of vibe coding's impact on organizational digital transformation and resilience. Through directed content analysis of 97 sources analyzed through Dynamic Capabilities, TOE, and DOI, we offer three principal contributions.

Addressing **RQ1**, we characterize vibe coding as the third paradigm in software development automation, document academic–practitioner discourse divergence, and provide a systematic comparative analysis extending Sanchis et al.'s (2020) framework.

Addressing **RQ2**, we identify and theorize the *"resilience paradox"*—vibe coding simultaneously enhances sensing, seizing, and transforming capabilities while introducing security vulnerabilities (45%), the "vulnerable developer" phenomenon (63%), technical debt, and skill erosion risk.

Addressing **RQ3**, we identify adoption factors through TOE analysis, characterize the DOI "quality chasm," and document four sustainability tensions: speed vs. quality, democratization vs. governance, innovation velocity vs. technical debt, and resource efficiency vs. environmental cost.

6.2. Practical Takeaways

For organizational leaders: (a) adopt a tiered governance framework matching rigor to criticality; (b) establish vibe engineering as the professional standard; (c) pursue hybrid development strategies; (d) invest in reskilling toward AI-augmented development; and (e) prepare for governance and maintenance challenges as vibe-coded applications mature.

6.3. Limitations

Several limitations must be acknowledged. First, the 12-month temporal scope captures a rapidly evolving phenomenon in its earliest stage. Second, English-only restriction excludes perspectives from non-English-speaking communities. Third, content analysis involves subjective interpretation despite intercoder reliability (Alpha = 0.83). Fourth, specific statistics may become outdated quickly. Fifth, our analysis draws on published discourse rather than primary organizational data.

6.4. Future Research Agenda

This study opens several avenues: (1) longitudinal case studies tracking resilience outcomes over time; (2) quantitative survey research testing our TOE model; (3) sector-specific deep dives in regulated industries; (4) environmental impact quantification of AI-assisted vs. traditional development; (5) focused investigation of the "vulnerable developer" phenomenon; (6) governance framework validation through action research; and (7) comparative international studies across regulatory and cultural contexts.

As vibe coding continues its rapid diffusion, the resilience paradox identified in this study underscores the urgency: vibe coding's extraordinary potential to accelerate digital transformation is matched by its capacity to introduce new fragilities. Organizations, policymakers, and researchers must engage critically with both dimensions to ensure this paradigm shift contributes to sustainable digital resilience rather than the accumulation of hidden digital risk.

References

Atos, 2025. Vibe Coding and the Future of Enterprise Development. Technical Report. Atos Digital Transformation Report.

Boston Consulting Group, 2025. From Potential to Profit: Closing the AI Impact Gap. Technical Report. BCG Henderson Institute.

Breidbach, C.F., Davern, M., Shanks, G., Tian, S., 2024. Orchestrating digital resilience: A socio-technical perspective. European Journal of Information Systems 33, 562–581.

Browder, R.E., 2024. Digital transformation as upgrading adaptation promoting enterprise resilience. Strategic Entrepreneurship Journal 18, 435–463.

Browder, R.E., Aldrich, H.E., Bradley, S.W., 2019. The emergence of the maker movement: Implications for entrepreneurship research. Journal of Business Venturing 34, 459–476.

Cabot, J., 2020. Low-code development and model-driven engineering: Two sides of the same coin? Software and Systems Modeling 19, 1–8.

Clancy, T., 2014. The Standish Group Chaos Report. Technical Report. Standish Group. West Yarmouth, MA.

CodeRabbit, 2025. The Security Impact of AI Co-Authored Code: A Quantitative Analysis. Research Report. CodeRabbit.

Collins Dictionary, 2025. Word of the year 2025: Vibe coding.

Elo, S., Kyngäs, H., 2008. The qualitative content analysis process. Journal of Advanced Nursing 62, 107–115. doi:10.1111/j.1365-2648.2007.04569.x.

Everest Group, 2024. The $2.3 Trillion Digital Transformation Challenge. Research Report. Everest Group.

Fawzy, A., Fahmy, M.A., Abdelfattah, A.S., 2025. Vibe coding in practice: A grey literature review. arXiv preprint arXiv:2510.00328 Accepted at ICSE 2026 SEIP Track.

Fryling, M., 2019. Low code app development. Journal of Computing Sciences in Colleges 34, 119.

Fuggetta, A., 1993. A classification of CASE technology. Computer Journal 35, 25–38. doi:10.1093/comjnl/35.1.25.

Garousi, V., Felderer, M., Mäntylä, M.V., 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. Information and Software Technology 106, 101–121. doi:10.1016/j.infsof.2018.09.006.

Gartner, 2025. Predicts 2025: Software Engineering. Technical Report. Gartner Research.

Genpact, 2025. Navigating the Vibe Coding Revolution: Enterprise Readiness Assessment. Technical Report. Genpact Insights.

Glover, N., Dudley, T., 1991. Practical Error Correction Design for Engineers. Data Systems Technology Corporation, Reston, VA.

HCLTech, 2025. AI-Powered Development: From Vibe Coding to Enterprise Value. Whitepaper. HCLTech.

Heeks, R., Ospina, A.V., 2019. Conceptualising the link between information systems and resilience: A developing country field study. Information Systems Journal 29, 70–96.

Holling, C.S., 1973. Resilience and stability of ecological systems. Annual Review of Ecology and Systematics 4, 1–23.

Hsieh, H.F., Shannon, S.E., 2005. Three approaches to qualitative content analysis. Qualitative Health Research 15, 1277–1288. doi:10.1177/1049732305276687.

Huff, C.C., 1992. Elements of a realistic CASE tool adoption budget. Communications of the ACM 35, 45–55. doi:10.1145/129617.129619.

Iivari, J., 1996. Why are CASE tools not used? Communications of the ACM 39, 94–103. doi:10.1145/236156.236183.

Imai, S., 2022. Is GitHub Copilot a substitute for human pair-programming?, in: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, pp. 38–42.

International Data Corporation, 1996. CASE Tools Market Report. Technical Report. IDC.

Karpathy, A., 2025. Vibe coding [X post]. X (formerly Twitter) post coining the term "vibe coding".

Krippendorff, K., 2019. Content Analysis: An Introduction to Its Methodology. 4th ed., Sage, Thousand Oaks, CA.

Lethbridge, C., 2024. Low-code is often no-code: How citizen developers are transforming enterprise IT. MIT Sloan Management Review .

Li, X., Wang, Y., 2024. Digital transformation and enterprise resilience: Evidence from China. PLOS ONE 19, e0298891.

Lincoln, Y.S., Guba, E.G., 1985. Naturalistic Inquiry. Sage, Newbury Park, CA.

Lundell, B., Lings, B., 2004. Changing perceptions of CASE technology. Journal of Systems and Software 72, 271–280. doi:10.1016/S0164-1212(03)00155-2.

Mayring, P., 2014. Qualitative Content Analysis: Theoretical Foundation, Basic Procedures and Software Solution. Sage.

McKinsey & Company, 2023. The agility advantage: How organizations can make decisions faster. McKinsey Quarterly .

McKinsey & Company, 2025. The State of AI: How Organizations Are Rewiring to Capture Value. Technical Report. McKinsey Global Survey.

Moore, G.A., 2014. Crossing the Chasm: Marketing and Selling Disruptive Products to Mainstream Customers. 3rd ed., Harper Business, New York.

Nicmanis, J., 2024. Reflexivity in qualitative content analysis: A guide for researchers. Qualitative Research 24, 89–107.

Orlikowski, W.J., 1993. CASE tools as organizational change: Investigating incremental and radical changes in systems development. MIS Quarterly 17, 309–340. doi:10.2307/249774.

OutSystems, 2019. The State of Application Development: Is IT Ready for Disruption? Technical Report. OutSystems. Boston, MA.

Pressman, R.S., Maxim, B.R., 2020. Software Engineering: A Practitioner's Approach. 9th ed., McGraw-Hill, New York.

Richardson, C., Rymer, J.R., 2014. New Development Platforms Emerge for Customer-Facing Applications. Technical Report. Forrester. Cambridge, MA.

Richardson, C., Rymer, J.R., 2016. Vendor Landscape: The Fractured, Fertile Terrain of Low-Code Application Platforms. Technical Report. Forrester. Cambridge, MA.

Rogers, E.M., 2003. Diffusion of Innovations. 5th ed., Free Press, New York.

Rymer, J.R., Koplowitz, R., 2019. The Forrester Wave: Low-Code Development Platforms for AD&D Professionals, Q1 2019. Technical Report. Forrester. Cambridge, MA.

Sahay, A., Indamutsa, A., Di Ruscio, D., Pierantonio, A., 2020. Supporting the understanding and comparison of low-code development platforms, in: Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE. pp. 171–178.

Sanchis, R., García-Perales, Ó., Fraile, F., Poler, R., 2020. Low-code as enabler of digital transformation in manufacturing industry. Applied Sciences 10, 12. doi:10.3390/app10010012.

Sanchis, R., Poler, R., 2019. Enterprise resilience assessment—a quantitative approach. Sustainability 11, 4327. doi:10.3390/su11164327.

Shmargad, J., 2025. Vibe Coding Comes to the Enterprise. Technical Report. Forrester Research.

Stack Overflow, 2025. 2025 Developer Survey Results. Technical Report. Stack Overflow.

Strubell, E., Ganesh, A., McCallum, A., 2019. Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 3645–3650.

Tabrizi, B., Lam, E., Girard, K., Irvin, V., 2019. Digital transformation is not about technology. Harvard Business Review March 13, 2019.

Teece, D.J., 2018. Business models and dynamic capabilities. Long Range Planning 51, 40–49.

Teece, D.J., Pisano, G., Shuen, A., 1997. Dynamic capabilities and strategic management. Strategic Management Journal 18, 509–533. doi:10.1002/(SICI)1097-0266(199708)18:7<509::AID-SMJ882>3.0.CO;2-Z.

Tisi, M., Mottu, J.M., Kolovos, D., De Lara, J., Guerra, E., Di Ruscio, D., Pierantonio, A., Wimmer, M., 2019. Lowcomote: Training the next generation of experts in scalable low-code engineering platforms. Available online: https://www.se.jku.at/lowcomote-training-the-next-generation-of-experts-in-scalable-low-code-engineer

Tornatzky, L.G., Fleischer, M., 1990. The Processes of Technological Innovation. Lexington Books, Lexington, MA.

Tyndall, D., 2010. AACODS checklist for appraising grey literature.

Various Authors, 2025a. r/programming: Vibe coding discussion threads. Reddit.

Various Authors, 2025b. Survey of vibe coding with large language models. arXiv preprint arXiv:2510.12399 .

Various Authors, 2025c. Vibe coding: AI-native paradigm. arXiv preprint arXiv:2510.17842 .

Various Authors, 2025d. Vibe coding: Democratization or disaster? DEV Community.

Various Authors, 2025e. Vibe coding: Flow, technical debt, and guidelines. arXiv preprint arXiv:2512.11922 .

Veracode, 2025. State of Software Security: The Rise of AI-Generated Code. Annual Report. Veracode.

Vial, G., 2019. Understanding digital transformation: A review and a research agenda. The Journal of Strategic Information Systems 28, 118–144. doi:10.1016/j.jsis.2019.01.003.

Warner, K.S.R., Wäger, M., 2019. Building dynamic capabilities for digital transformation: An ongoing process of strategic renewal. Long Range Planning 52, 326–349. doi:10.1016/j.lrp.2018.12.001.

Willison, S., 2025. Not all AI-assisted programming is vibe coding. Simon Willison's Weblog.

Y Combinator, 2025. W2025 batch demographics and technology trends. Y Combinator Blog.

Ziegler, A., Kalliamvakou, E., Li, X.A., Rice, A., Rifkin, D., Simister, S., Sittampalam, G., Aftandilian, E., 2022. Productivity assessment of neural code completion, in: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, pp. 21–29.